EMMAeye V3.1: Comprehensive Technical Documentation

CONTENT TABLE

1. Project Overview

EMMAeye is a biometric analysis system that leverages multiple concurrent Convolutional Neural Networks (CNNs). The system is designed to perform four primary tasks in real-time:

- **Pose Estimation:** Localizing 33 human body keypoints.
- **Persistent Tracking:** Assigning and maintaining unique IDs to individuals across frames.
- **Hand Gesture Recognition:** Identifying specific gestures (G1–G16) based on a predefined reference matrix.
- **Data Association:** Intelligently linking hands and faces to specific bodies using Euclidean proximity and adaptive thresholds**.**

2. Environment & System Requirements

To ensure system stability, the following technical specifications must be met:

- **Operating System:** Windows 10 or 11 (64-bit).
- **Language:** Python 3.9.x is recommended for optimal MediaPipe compatibility.
- **Core Libraries:**
    - **opencv-python (4.8.1.78):** For image processing and UI rendering.
    - **mediapipe (0.10.14):** For pose landmarker model inference.
    - **cvzone (1.6.1):** For hand detection and segmentation.
    - **numpy (1.24.4):** For matrix operations and linear algebra.
    - **protobuf==3.20.3:** Explicitly downgraded to prevent descriptor initialization crashes inherent in newer MediaPipe builds.
    - **paho-mqtt==1.6.1:** Required for network telemetry broadcasting to external MQTT brokers.

Required Assets (Models)

The project requires the following binary files in the root directory:

1. **pose_landmarker_lite.task:** A compiled machine learning model bundle provided by Google's MediaPipe. It contains the pre-trained neural network weights required to detect human bodies and map out 33 skeletal landmarks (such as shoulders, elbows, and wrists) in real-time. The "lite" designation indicates that the mathematics have been compressed and optimized to run efficiently on standard CPUs without requiring dedicated GPU hardware.
2. **res10_300x300_ssd_iter_140000.caffemodel:** A pre-trained Deep Neural Network (DNN) weights file built on the Caffe framework. This file stores the optimized weights and feature representations required for facial bounding box regression. It utilizes a ResNet-10 architecture configured as a Single Shot Detector (SSD). During execution, the face_detector_dnn.py module loads these numerical weights into memory to **process the camera feed and generate facial bounding boxes.**
3. **deploy.prototxt:** A text-based configuration architecture file. This acts as the structural blueprint for the .caffemodel. It defines the layer connectivity and data flow parameters for the neural network. The OpenCV DNN module requires both the blueprint (.prototxt) and the weights (.caffemodel) to successfully initialize the face detection model.

3. Modular Architecture

- **main_emmaeye.py (Primary Execution Pipeline):** The entry point of the application. It manages the video stream lifecycle and coordinates the execution of all sub-modules within a high-frequency Main Loop. It also handles timestamp synchronization to prevent inference engine collisions.
- **person.py (Data Structures):** Defines the system's Dataclasses:
    - **PoseData:** Stores (x, y) coordinates and visibility scores for body keypoints.
    - **HandData:** Contains 21 hand landmarks, bounding boxes, and gesture states.
    - **Person:** The primary entity class that unifies pose, face, and hand data under a single persistent ID.
- **tracker.py (Persistent Tracking):** Implements a Greedy Matching algorithm. It calculates the distance between a person's last known center and current detections. If the distance is within the match_thresh_px (scaled by shoulder distance), the ID is maintained.
- **associator.py (Sensor Fusion Logic):** The analytical module responsible for correlating independent detections:
    1. **Hand-to-Wrist Coupling:** Uses an adaptive threshold based on the subject's shoulder distance to assign hands to the correct body side.
    2. **Gesture Stabilization:** Utilizes a GestureStabilizer with a voting queue. A gesture is only "confirmed" after appearing in a set number of frames to prevent visual jitter.
- **logger_jsonl.py (Data Persistence):** Records all telemetry in JSONL (JSON Lines) format. It includes a CustomEncoder to safely serialize complex Numpy data types (like int64), ensuring data integrity for Big Data analysis.

4. Key Technical Concepts

- **Gesture Matrix (GESTURE_MATRIX):** A binary matrix where rows represent specific gestures and columns represent finger state features.
- **Euclidean Distance:** The mathematical method used to calculate the linear separation between points in the video's Cartesian plane.
- **Adaptive Thresholding:** Instead of fixed pixel values, the system scales detection sensitivity based on the shoulder_dist. This ensures accuracy regardless of whether the subject is near or far from the camera.
- **Monotonic Timestamps:** The system enforces strictly increasing timestamps (t_ms) to prevent MediaPipe from crashing on identical millisecond reads.

5. Data Dictionary (JSONL Output Telemetry)

The telemetry output is serialized into the emmaeye_log.jsonl database frame-by-frame. Each line constitutes a strictly formatted, standalone JSON object representing a single temporal snapshot of the camera feed.

Root Variables (Temporal Synchronization)

These variables define the exact moment of inference for the entire frame buffer.

- **timestamp_ms *(Integer)*:** The standard UNIX epoch time in milliseconds. Used for absolute chronological logging.
- **ms_after_midnight *(Integer)*:** The precise temporal offset in milliseconds elapsed since 00:00:00 of the current local day. This serves as the primary synchronization metric for correlating camera telemetry with external network systems.
- **people *(Array of Objects)*:** A dynamically sized array containing all successfully tracked human entities within the current frame. If no humans are detected, this array remains empty.

Entity Variables (Person Object)

Each object within the people array encapsulates the complete biometric state of a single individual.

- **id *(Integer)*:** The persistent tracking identifier assigned by the Greedy Matching

algorithm. This value remains constant for an individual across multiple frames.

Posture Sub-Object (pose)

Contains the spatial data derived from the MediaPipe pose landmarker.

- **landmarks *(2D Array [33x3] of Floats)*:** The complete skeletal matrix. Each row corresponds to a specific anatomical joint (indices 0-32). The three columns represent:
    1. **X** coordinate (normalized or pixel-mapped depending on configuration).
    2. **Y** coordinate (normalized or pixel-mapped).
    3. Visibility score (**0.0** to **1.0),** indicating the algorithmic certainty that the joint is unoccluded and present within the physical frame.
- **center *(Array [2] of Floats)*:** The calculated structural centroid of the upper body, formatted as **[X, Y].** Utilized by the temporal tracker for Euclidean distance matching across frames.
- **shoulder_dist *(Float)*:** The linear Cartesian distance in pixels between the left and right shoulder landmarks. This acts as the dynamic anatomical ruler for the subject.
- **coupling_thresh *(Float)*:** The calculated maximum allowable pixel radius for hand-to-wrist association. Derived directly from the shoulder_dist to ensure adaptive spatial scaling.

Facial Recognition Sub-Object (face)

Contains the bounding box logic derived from the **Deep Neural Network (DNN).**

- **bbox *(Array [4] of Integers/Floats)*:** The spatial boundaries of the detected face, formatted strictly as **[X_min, Y_min, Width, Height].**
- **conf *(Float)*:** The statistical confidence score (0.0 to 1.0) output by the ResNet-10 SSD architecture, indicating the probability of a true positive facial detection.

Hand Tracking Sub-Object (hands)

A compartmentalized object separating data into specific lateral appendages: BODY_LEFT and BODY_RIGHT. If a specific hand is not detected or successfully associated, its respective nested object will be null or omitted.

- **BODY_LEFT / BODY_RIGHT *(Objects)*:**
    - **lm *(2D Array [21x3] of Floats)*:** The localized matrix of hand joints. The columns represent the X, Y, and Z (depth approximation) coordinates for each specific knuckle and fingertip.
    - **bbox *(Array [4] of Integers/Floats)*:** The spatial boundaries of the hand, formatted as [X_min, Y_min, Width, Height].
    - **raw_gesture *(Integer, 1-16)*:** The immediate, unfiltered classification output derived from computing the Hamming distance against the gesture matrix for the current frame.
    - **stable_gesture *(Integer, 1-16)*:** The verified classification output after passing through the temporal voting queue. This is the noise-filtered gesture state.
    - **dist_to_wrist *(Float)*:** The absolute Euclidean error in pixels between the hand's origin coordinate and the assigned skeletal wrist. This proves the mathematical validity of the spatial coupling algorithm.

6. Quick Start Guide

- **Enviroment:** Make sure you are using python 9.3.X to avoid compatibility problems.
- **Installation:** Run pip install -r install.txt.
- **Execution**: Launch the main execution pipeline via python main_emmaeye.py.
- **Controls:**
    - The UI displays green bounding boxes for faces and magenta labels for gestures.
    - Press ESC to safely terminate the session and flush logs to disk.
- **Output:** Consult emmaeye_log.jsonl for a full technical breakdown of the recorded session.
-

7.main_emmaeye.py

Role: Primary Execution Pipeline and Hardware Interface

Explicit Description: This module operates as the central control structure of the entire software architecture. Its execution is strictly divided into three distinct operational phases:

1. **Initialization Sequence:** The module begins by establishing a connection with the local video capture peripheral, configuring it to the required high-definition resolution (1280x720) to ensure sufficient pixel density for distant feature extraction. Concurrently, it loads the required machine learning models into system memory. This involves instantiating the MediaPipe Pose landmarker, the CVZone Hand Detector, and reading the .prototxt and .caffemodel files to initialize the OpenCV Deep Neural Network (DNN) for facial recognition. Finally, it initializes the local logging objects and the external network communication threads.
2. **Synchronous Inference Loop:** This is the high-frequency execution loop that processes the video feed frame-by-frame. For every frame acquired, the module performs essential image preprocessing, such as BGR to RGB color space conversion. It generates a strictly monotonic timestamp (t_ms) to pass into the MediaPipe engine, which is a critical step to prevent backend collisions caused by processing frames too quickly. The module then commands the sequential execution of the pose, face, and hand models.
3. **Data Dispatch and Resource Management:** The raw coordinate data generated by the three models is dispatched to the analytical modules (tracker.py and associator.py). Once these modules return the fully structured Person array, main_emmaeye.py triggers the telemetry protocols, calling the JSONL logger to serialize the data to disk and the network publisher to broadcast it. Finally, it overlays the visual data (bounding boxes, skeletal lines, and gesture labels) onto the frame buffer for real-time monitoring. The loop listens continuously for an interrupt signal (the ESC key), triggering a graceful cleanup phase that releases the camera hardware, closes network ports, and flushes all memory buffers to prevent data corruption**.**

8. person.py

Role: Data Structures and Entity Encapsulation

Explicit Description: This module establishes the foundational data schemas and object-oriented architecture for the entire system. Instead of relying on disparate lists and global variables to manage spatial coordinates, this file defines the strictly typed Dataclasses responsible for state encapsulation.

It specifies three primary sub-structures:

1. **PoseData**: Engineered to store the normalized (x, y) coordinate arrays and visibility scores for the 33 body keypoints extracted by the MediaPipe framework. It also calculates and stores critical derived metrics, such as the anatomical center of mass and the dynamic shoulder distance used for adaptive thresholding.
2. **HandData**: Designed to contain the 21-point hand landmark matrices, localized bounding boxes, and the discrete gesture classification states (both raw and stabilized).
3. **Person**: The primary hierarchical entity class. It acts as a unified container that aggregates the PoseData, HandData, and facial bounding box arrays under a single, persistent identification number.

By encapsulating these disparate biometric inputs into a singular Person object, the module ensures strict data integrity and system scalability. This design pattern allows the execution pipeline to process and track an arbitrary number of individuals concurrently without the risk of variable collision or cross-contamination of sensor data in multi-user scenarios.

9. tracker.py

Role: Persistent Temporal Tracking and Identity Management

Explicit Description: This module is responsible for maintaining the temporal continuity of detected entities across sequential video frames. To achieve this, it implements a Greedy Matching algorithm based on Euclidean distance heuristics. By calculating the spatial displacement between a person's historical center of mass and newly detected anatomical

keypoints, the tracker deterministically assigns and preserves a unique, persistent identification number (ID) for each individual within the camera's field of view.

To mitigate identity swapping or premature ID reassignment during temporary visual occlusions or tracking failures, the module utilizes a frame-buffer memory system. If an entity is lost, the tracker retains its ID in memory up to a predefined maximum threshold of missed frames before permanently purging it from the active tracking registry. Furthermore, the distance threshold (match_thresh_px) utilized for identity association is dynamically scaled using the anatomical shoulder distance of the subject. This adaptive scaling ensures highly robust tracking performance regardless of the user's physical proximity to or distance from the camera sensor.

10. associator.py

Role: Sensor Fusion Logic and Gesture Classification

Explicit Description: This module functions as the analytical core responsible for correlating independent biometric detections. It executes two primary algorithms to ensure data integrity and classification accuracy:

1. **Spatial Coupling (Hand-to-Wrist Association):** To resolve the spatial ambiguity of unlinked anatomical parts, the module applies an adaptive threshold derived directly from the subject's calculated shoulder distance. By evaluating the Euclidean proximity between detected hand bounding boxes and the skeletal wrist coordinates, it deterministically binds the hand data to the correct lateral side (Left or Right) of the specific Person entity. This dynamic scaling approach prevents false-positive associations in multi-user environments, ensuring that limbs are mapped correctly even when users overlap within the camera's field of view.
2. **Gesture Matching and Stabilization:** The module classifies raw finger configurations by computing the Hamming distance against a predefined, binary GESTURE_MATRIX (representing gestures G1-G16). To mitigate high-frequency classification noise, it subsequently processes the raw inference outputs through a GestureStabilizer algorithm. This sub-component utilizes a temporal voting queue, ensuring a specific gesture state is only confirmed and integrated into the telemetry stream after it achieves numerical dominance across a continuous sequence of recent frames, thereby eliminating visual jitter.

11. face_detector_dnn.py

Role: Deep Neural Network Facial Recognition and Localization

Explicit Description: This module operates as a dedicated computational wrapper for the OpenCV Deep Neural Network (DNN) inference engine. It is specifically engineered to instantiate and execute a pre-trained ResNet-10 architecture, mathematically configured as a Single Shot Detector (SSD).

During the initialization phase, the module loads the structural network blueprint from the .prototxt file and the optimized numerical weights from the .caffemodel file. During the active synchronous loop, it processes the raw image matrices to identify human facial features, abstracting the complexities of the underlying Caffe framework. The algorithm outputs precise Cartesian bounding box arrays [x, y, w, h] alongside statistical confidence scores (conf). These outputs are subsequently dispatched to the primary execution pipeline to be integrated into the unified Person entity, facilitating comprehensive biometric tracking.

12. logger_jsonl.py

Role: Data Persistence and Telemetry Serialization

Explicit Description: This module is responsible for the robust serialization and permanent storage of the system's biometric telemetry. It utilizes a JSON Lines (.jsonl) architecture to continuously append high-frequency, frame-by-frame entity data directly to the local storage drive.

A highly critical component of this module is the implementation of a CustomEncoder class. This algorithmic encoder safely intercepts and parses complex, non-native data structures—specifically multi-dimensional numpy matrices and int64 scalars—converting them into standard JSON-compliant formats. This process ensures absolute structural integrity for

downstream Big Data analysis and fundamentally prevents type-casting errors or runtime segmentation faults during execution. Furthermore, the logging protocol enforces immediate buffer flushing to the emmaeye_log.jsonl database, guaranteeing that all spatial coordinates, gesture classifications, and synchronization timestamps (ms_after_midnight) are safely preserved, even in the event of an unexpected hardware interruption or software termination.